



The seL4[®] Foundation

<https://sel4.foundation>

Microkit User Manual (v1.4.1)

Contents

1	Introduction	1
1.1	Purpose	1
1.2	Overview	1
1.3	Document Structure	2
2	Concepts	3
2.1	System	3
2.2	Protection Domains	3
2.2.1	Entry points	3
2.2.2	Scheduling	4
2.3	Virtual Machine	4
2.4	Memory Regions	5
2.5	Channels	5
2.5.1	Protected procedure	5
2.5.2	Notification	6
2.6	Interrupt	6
2.7	Fault	7
3	SDK	8
3.1	Configurations	8
3.2	Debug	8
3.3	Release	8
3.4	Benchmark	8
3.5	System Requirements	8
4	Microkit Tool	10
5	libmicrokit	11
5.1	void init(void)	11
5.2	void notified(microkit_channel ch)	11
5.3	microkit_msginfo protected(microkit_channel ch, microkit_msginfo msginfo)	12
5.4	seL4_Bool fault(microkit_child child, microkit_msginfo msginfo, microkit_msginfo *reply_msginfo)	12
5.5	microkit_msginfo microkit_ppcall(microkit_channel ch, microkit_msginfo msginfo)	12
5.6	void microkit_notify(microkit_channel ch)	12
5.7	void microkit_irq_ack(microkit_channel ch)	13
5.8	void microkit_deferred_notify(microkit_channel ch)	13
5.9	void microkit_deferred_irq_ack(microkit_channel ch)	13
5.10	void microkit_pd_restart(microkit_child pd, uintptr_t entry_point)	13
5.11	void microkit_pd_stop(microkit_child pd)	13
5.12	microkit_msginfo microkit_msginfo_new(uint64_t label, uint16_t count)	13
5.13	uint64_t microkit_msginfo_get_label(microkit_msginfo msginfo)	13
5.14	uint64_t microkit_msginfo_get_count(microkit_msginfo msginfo)	13
5.15	uint64_t microkit_mr_get(uint8_t mr)	13
5.16	void microkit_mr_set(uint8_t mr, uint64_t value)	13
5.17	void microkit_vcpu_restart(microkit_child vcpu, seL4_Word entry_point)	14
5.18	void microkit_vcpu_stop(microkit_child vcpu)	14

5.19	<code>void microkit_vcpu_arm_inject_irq(microkit_child vcpu, seL4_Uint16 irq, seL4_Uint8 priority, seL4_Uint8 group, seL4_Uint8 index)</code>	14
5.20	<code>void microkit_vcpu_arm_ack_vppi(microkit_child vcpu, seL4_Word irq)</code>	14
5.21	<code>seL4_Word microkit_vcpu_arm_read_reg(microkit_child vcpu, seL4_Word reg)</code>	14
5.22	<code>void microkit_vcpu_arm_write_reg(microkit_child vcpu, seL4_Word reg, seL4_Word value)</code>	14
6	System Description File	15
6.1	<code>protection_domain</code>	15
6.2	<code>memory_region</code>	16
6.2.1	Page sizes by architecture	16
6.3	<code>channel</code>	17
7	Board Support Packages	18
7.1	i.MX8MM-EVK	18
7.2	i.MX8MQ-EVK	18
7.3	MaaXBoard	18
7.4	Odroid-C2	18
7.5	Odroid-C4	18
7.6	TQMa8XQP 1GB	18
7.7	QEMU virt (AArch64)	19
7.8	QEMU virt (RISC-V 64-bit)	20
7.9	Pine64 Star64	20
7.10	ZCU102	20
7.11	Adding Platform Support	21
7.11.1	Prerequisites	21
7.11.2	Getting Microkit Components Working	21
7.11.3	Contributing Platform Support	21
8	Rationale	22
8.1	Overview	22
8.2	Protection Domains	22
8.3	Protected Procedure Priorities	22
8.4	Protected Procedure Argument Size	22
8.5	Limits	23

1 Introduction

The seL4 Microkit is a small and simple operating system (OS) built on the seL4 microkernel. Microkit is designed for building system with a *static architecture*. A static architecture is one where system resources are assigned up-front at system initialisation time.

1.1 Purpose

The Microkit is intended to:

- provide a small and simple OS for a wide range of IoT, cyberphysical and other embedded use cases;
- provide a reasonable degree of application portability appropriate for the targeted use cases;
- make seL4-based systems easy to develop and deploy within the target areas;
- leverage seL4's strong isolation properties to support a near-minimal *trusted computing base* (TCB);
- retain seL4's trademark performance for systems built with it;
- be, in principle, amenable to formal analysis of system safety and security properties (although such analysis is beyond the initial scope).

1.2 Overview

A Microkit system is built from a set of individual programs that are isolated from each other, and the system, in *protection domains*. Protection domains can interact by calling *protected procedures* or sending *notifications*.

Microkit is distributed as a software development kit (SDK). The SDK includes the tools, libraries and binaries required to build a Microkit system. The Microkit source is also available which allows you to customize or extend Microkit and produce your own SDK.

To build a Microkit system you will write some programs that use `libmicrokit`. Microkit programs are a little different to a typical process on a Linux-like operating system. Rather than a single `main` entry point, a program has four distinct entry points: `init`, `notified` and, potentially, `protected`, `fault`.

The individual programs are combined to produce a single bootable *system image*. The format of the image is suitable for loading by the target board's bootloader. The Microkit tool, which is provided as part of the SDK, is used to generate the system image.

The Microkit tool takes a *system description* as input. The system description is an XML file that specifies all the objects that make up the system.

Note: Microkit does **not** impose any specific build system; you are free to choose the build system that works best for you.

1.3 Document Structure

The **Concepts** chapter describes the various concepts that make up Microkit. It is recommended that you familiarise yourself with these concepts before trying to build a system.

The **SDK** chapter describes the software development kit, including its components and system requirements for use.

The **Microkit tool** chapter describes the host system tool used for generating a system image from the system description and user-programs.

The **libmicrokit** chapter describes the interfaces to the Microkit library.

The **System Description File** chapter describes the format of the system description XML file.

The **Board Support Packages** chapter describes each of the board support packages included in the SDK.

The **Rationale** chapter documents the rationale for some of the key design choices of in Microkit.

2 Concepts

This chapter describes the key concepts provided by Microkit.

As with any set of concepts there are words that take on special meanings. This document attempts to clearly describe all of these terms, however as the concepts are inter-related it is sometimes necessary to use a term prior to its formal introduction.

- **system**
- **protection domain (PD)**
- **virtual machine (VM)**
- **memory region**
- **channel**
- **protected procedure**
- **notification**
- **interrupt**
- **fault**

2.1 System

At the most basic level Microkit provides the platform for running a *system* on a specific board. As a *user* of Microkit you use the platform to create a software system that implements your use case. The system is described in a declarative configuration file, and the Microkit tool takes this system description as an input and produces an appropriate system image that can be loaded on the target board.

The key elements that make up a system are *protection domains*, *memory regions* and *channels*.

2.2 Protection Domains

A **protection domain** (PD) is the fundamental runtime abstraction in Microkit. It is analogous, but very different in detail, to a process on a UNIX system.

A PD provides a thread of control that executes within a fixed virtual address space. The isolation provided by the virtual address space is enforced by the underlying hardware MMU.

The virtual address space for a PD has mappings for the PD's *program image* along with any memory regions that the PD can access. The program image is an ELF file containing the code and data which implements the isolated component.

Microkit supports a maximum of 63 protection domains.

2.2.1 Entry points

Although a protection domain is somewhat analogous to a process, it has a considerably different program structure and life-cycle. A process on a typical operating system will have a `main` function which is invoked by the system when the process is created. When the `main` function returns the process is destroyed.

By comparison a protection domain has up to four entry points: `*init`, `notified` which are required. `*protected` which is optional. `*fault` which is required if the PD has children.

When a Microkit system is booted, all PDs in the system execute the `init` entry point.

The `notified` entry point will be invoked whenever the protection domain receives a *notification* on a *channel*. The `protected` entry point is invoked when a PD's *protected procedure* is called by

another PD. A PD does not have to provide a protected procedure, therefore the `protected` entry point is optional.

The `fault` entry point is invoked when a PD that is a child of another PD causes a fault. A PD does not have to have child PDs, therefore the `fault` entry point is only required for a parent PD.

These entry points are described in more detail in subsequent sections.

Note: The processing of `init` entry points is **not** synchronised across protection domains. Specifically, it is possible for a high priority PD's `notified` or `protected` entry point to be called prior to the completion of a low priority PD's `init` entry point.

The overall computational model for a Microkit system is a set of isolated components reacting to incoming events.

2.2.2 Scheduling

The PD has a number of scheduling attributes that are configured in the system description:

- `priority` (0 – 254)
- `period` (microseconds)
- `budget` (microseconds)
- `passive` (boolean)

The `budget` and `period` bound the fraction of CPU time that a PD can consume. Specifically, the **budget** specifies the amount of time for which the PD is allowed to execute. Once the PD has consumed its budget, it is no longer runnable until the budget is replenished; replenishment happens once every **period** and resets the budget to its initial value. This means that the maximum fraction of CPU time the PD can consume is `budget/period`.

The budget cannot be larger than the period. A budget that equals the period (aka. a “full” budget) behaves like a traditional time slice: After executing for a full period, the PD is preempted and put at the end of the scheduling queue of its priority. In other words, PDs with equal priorities and full budgets are scheduled round-robin with a time slice defined by the period.

The **priority** determines which of the runnable PDs to schedule. A PD is runnable if one of its entry points has been invoked and it has budget remaining in the current period. Runnable PDs of the same priority are scheduled in a round-robin manner.

The **passive** determines whether the PD is passive. A passive PD will have its scheduling context revoked after initialisation and then bound instead to the PD's notification object. This means the PD will be scheduled on receiving a notification, whereby it will run on the notification's scheduling context. When the PD receives a *protected procedure* by another PD or a *fault* caused by a child PD, the passive PD will run on the scheduling context of the callee.

2.3 Virtual Machine

A *virtual machine* (VM) is a runtime abstraction for running guest operating systems in Microkit. It is similar to a protection domain in that it provides a thread of control that executes within an isolated virtual address space.

The main difference between a VM and a PD is that VMs have a higher privilege level such that they may function as a guest operations and have their own user-space programs at a separate exception level.

The virtual machine is always a child of a PD. Exceptions caused by the virtual machine are delivered to the parent PD through the `fault` entry point. Each virtual machine has a ‘virtual CPU’

associated with it which is used to identify the fault. At the moment, all VMs only have a single virtual CPU but in the future multi-vCPU VMs will be allowed.

The parent PD is responsible for starting and managing the virtual machine. Microkit provides the abstractions in order to manage the virtual machine through seL4 but there is typically a non-trivial amount of supporting code/infrastructure to properly start and manage a VM.

To keep the (potentially untrusted) virtual machine isolated from the rest of the system, Microkit enforces that a protection domain can only ever manage a single virtual machine.

2.4 Memory Regions

A *memory region* is a contiguous range of physical memory. A memory region may have a *fixed* physical address. For memory regions without a fixed physical address, the physical address is allocated as part of the build process. Typically, memory regions with a fixed physical address represent memory-mapped device registers.

Memory regions that are within main memory are zero-initialised.

The size of a memory region must be a multiple of a supported page size. The supported page sizes are architecture dependent. For example, on AArch64 architectures, Microkit support 4KiB and 2MiB pages. The page size for a memory region may be specified explicitly in the system description. If page size is not specified, the smallest supported page size is used.

Note: The page size also restricts the alignment of the memory region's physical address. A fixed physical address must be a multiple of the specified page size.

A memory region can be *mapped* into one or more protection domains. The mapping has a number of attributes, which include:

- the virtual address at which the region is mapped in the PD
- caching attributes (mostly relevant for device memory)
- permissions (read, write and execute)

Note: When a memory region is mapped into multiple protection domains, the attributes used for different mappings may vary.

2.5 Channels

A *channel* enables two protection domains to interact using protected procedures or notifications. Each connects exactly two PDs; there are no multi-party channels.

When a channel is created between two PDs, a *channel identifier* is configured for each PD. The *channel identifier* is used by the PD to reference the channel. Each PD can refer to the channel with a different identifier. For example if PDs **A** and **B** are connected by a channel, **A** may refer to the channel using an identifier of **37** while **B** may use **42** to refer to the same channel.

Note: There is no way for a PD to directly refer to another PD in the system. PDs can only refer to other PDs indirectly if there is a channel between them. In this case, the channel identifier is effectively a proxy identifier for the other PD. So, to extend the prior example, **A** can indirectly refer to **B** via the channel identifier **37**. Similarly, **B** can refer to **A** via the channel identifier **42**.

The system supports a maximum of 63 channels and interrupts per protection domain.

2.5.1 Protected procedure

A protection domain may provide a *protected procedure* (PP) which can be invoked from another protection domain. Up to 64 words of data may be passed as arguments when calling a protected

procedure. The protected procedure return value may also be up to 64 words.

When a protection domain calls a protected procedure, the procedure executes within the context of the providing protection domain.

A protected call is only possible if the callee has strictly higher priority than the caller. Transitive calls are possible, and as such a PD may call a *protected procedure* in another PD from a protected entry point. However the overall call graph between PDs must form a directed, acyclic graph. It follows that a PD can not call itself, even indirectly. For example, A calls B calls C is valid (subject to the priority constraint), while A calls B calls A is not valid.

When a protection domain is called, the protected entry point is invoked. The control returns to the caller when the protected entry point returns.

The caller is blocked until the callee returns. Protected procedures must execute in bounded time. It is intended that a future version of Microkit will enforce this condition through static analysis. In the present version the caller must trust the callee to conform.

In general, PPs are provided by services for use by clients that trust the protection domain to provide that service.

To call a PP, a PD calls `microkit_ppcall` passing the channel identifier and a *message* structure. A *message* structure is returned from this function.

When a PD's protected procedure is invoked, the protected entry point is invoked with the channel identifier and message structure passed as arguments. The protected entry point must return a message structure.

2.5.2 Notification

A notification is a (binary) semaphore-like synchronisation mechanism. A PD can *notify* another PD to indicate availability of data in a shared memory region if they share a channel.

To notify another PD, a PD calls `microkit_notify`, passing the channel identifier. When a PD receives a notification, the notified entry point is invoked with the appropriate channel identifier passed as an argument.

Unlike protected procedures, notifications can be sent in either direction on a channel regardless of priority.

Note: Notifications provide a mechanism for synchronisation between PDs, however this is not a blocking operation. If a PD notifies another PD, that PD will become scheduled to run (if it is not already), but the current PD does **not** block. Of course, if the notified PD has a higher priority than the current PD, then the current PD will be preempted (but not blocked) by the other PD.

2.6 Interrupt

Hardware interrupts can be used to notify a protection domain. The system description specifies if a protection domain receives notifications for any hardware interrupt sources. Each hardware interrupt is assigned a channel identifier. In this way the protection domain can distinguish the hardware interrupt from other notifications. A specific hardware interrupt can only be associated with at most one protection domain. It should be noted that once a hardware interrupt has been received, it will not be received again until `microkit_irq_ack` is called. The seL4 kernel will mask the hardware interrupt until it has been acknowledged.

Microkit does not provide timers, nor any *sleep* API. After initialisation, activity in the system is initiated by an interrupt causing a notified entry point to be invoked. That notified function may in turn notify or call other protection domains that cause other system activity, but eventually all

activity indirectly initiated from that interrupt will complete, at which point the system is inactive again until another interrupt occurs.

2.7 Fault

Faults such as an invalid memory access or illegal instruction are delivered to the seL4 kernel which then forwards them to a designated 'fault handler'. By default, all faults caused by protection domains go to the system fault handler which simply prints out details about the fault in a debug configuration.

When a protection domain is a child of another protection domain, the designated fault handler for the child is the parent protection domain. The same applies for a virtual machine.

This means that whenever a fault is caused by a child, it will be delivered to the parent PD instead of the system fault handler via the `fault` entry point. It is then up to the parent to decide how the fault is handled.

3 SDK

Microkit is distributed as a software development kit (SDK).

The SDK includes support for one or more *boards*. Three *configurations* are supported for each board: *debug*, *release*, and *benchmark*. See [the Configurations section](#) for more details.

The SDK contains:

- Microkit user manual (this document)
- Microkit tool

Additionally, for each supported board configuration the following are provided:

- `libmicrokit`
- `loader.elf`
- `kernel.elf`
- `monitor.elf`

For some boards there are also examples provided in the `examples` directory.

The Microkit SDK does **not** provide, nor require, any specific build system. The user is free to build their system using whatever build system is deemed most appropriate for their specific use case.

The Microkit tool should be invoked by the system build process to transform a system description (and any referenced program images) into an image file which can be loaded by the target board's bootloader.

The ELF files provided as program images should be standard ELF files and have been linked against the provided `libmicrokit`.

3.1 Configurations

3.2 Debug

The *debug* configuration includes a debug build of the seL4 kernel to allow console debug output using the kernel's UART driver.

3.3 Release

The *release* configuration is a release build of the seL4 kernel and is intended for production builds. The loader, monitor, and kernel do *not* perform any serial output.

3.4 Benchmark

The *benchmark* configuration uses a build of the seL4 kernel that exports the hardware's performance monitoring unit (PMU) to PDs. The kernel also tracks information about CPU utilisation. This benchmark configuration exists due a limitation of the seL4 kernel and is intended to be removed once [RFC-16 is implemented](#).

3.5 System Requirements

The Microkit tool requires Linux (x86-64), macOS (x86-64 or AArch64).

On Linux, the Microkit tool is statically linked and should run on any distribution.

On macOS, the Microkit tool should run on macOS 10.12 (Sierra) or higher.

The Microkit tool does not depend on any additional system binaries.

4 Microkit Tool

The Microkit tool is available in `bin/microkit`.

The Microkit tool takes as input a system description. The format of the system description is described in a subsequent chapter.

Usage:

```
microkit [-h] [-o OUTPUT] [-r REPORT] --board [BOARD] --config CONFIG  
        [--search-path [SEARCH_PATH ...]] system
```

The path to the system description file, board to build the system for, and configuration to build for must be provided.

The search paths provided tell the tool where to find any program images specified in the system description file.

In the case of errors, a diagnostic message shall be output to `stderr` and a non-zero code returned.

In the case of success, a loadable image file and a report shall be produced. The output paths for these can be specified by `-o` and `-r` respectively. The default output paths are `loader.img` and `report.txt`.

The loadable image will be a binary that can be loaded by the board's bootloader.

The report is a plain text file describing important information about the system. The report can be useful when debugging potential system problems. This report does not have a fixed format and may change between versions. It is not intended to be machine readable.

5 libmicrokit

All program images should link against `libmicrokit.a`.

The library provides the C runtime for the protection domain, along with interfaces for the Microkit APIs.

The component must provide the following functions:

```
void init(void);
void notified(microkit_channel ch);
```

If the protection domain provides a protected procedure it must also implement:

```
microkit_msginfo protected(microkit_channel ch, microkit_msginfo msginfo);
```

If the protection domain has children it must also implement:

```
seL4_Bool fault(microkit_child child, microkit_msginfo msginfo,
                microkit_msginfo *reply_msginfo);
```

libmicrokit provides the following functions:

```
microkit_msginfo microkit_ppcall(microkit_channel ch, microkit_msginfo msginfo);
void microkit_notify(microkit_channel ch);
microkit_msginfo microkit_msginfo_new(seL4_Word label, seL4_Uint16 count);
seL4_Word microkit_msginfo_get_label(microkit_msginfo msginfo);
seL4_Word microkit_msginfo_get_count(microkit_msginfo msginfo);
void microkit_irq_ack(microkit_channel ch);
void microkit_deferred_notify(microkit_channel ch);
void microkit_deferred_irq_ack(microkit_channel ch);
void microkit_pd_restart(microkit_child pd, seL4_Word entry_point);
void microkit_pd_stop(microkit_child pd);
void microkit_mr_set(seL4_Uint8 mr, seL4_Word value);
seL4_Word microkit_mr_get(seL4_Uint8 mr);
void microkit_vcpu_restart(microkit_child vcpu, seL4_Word entry_point);
void microkit_vcpu_stop(microkit_child vcpu);
void microkit_vcpu_arm_inject_irq(microkit_child vcpu, seL4_Uint16 irq,
                                  seL4_Uint8 priority, seL4_Uint8 group,
                                  seL4_Uint8 index);
void microkit_vcpu_arm_ack_vppi(microkit_child vcpu, seL4_Word irq);
seL4_Word microkit_vcpu_arm_read_reg(microkit_child vcpu, seL4_Word reg);
void microkit_vcpu_arm_write_reg(microkit_child vcpu, seL4_Word reg, seL4_Word value);
```

5.1 void init(void)

Every PD must expose an `init` entry point. This is called by the system at boot time.

5.2 void notified(microkit_channel ch)

The `notified` entry point is called by the system when a PD has received a notification on a channel.

`ch` identifies the channel which has been notified (and indirectly the PD that performed the notification).

Note: `ch` could identify an interrupt.

Channel identifiers are specified in the system configuration.

5.3 `microkit_msginfo protected(microkit_channel ch, microkit_msginfo msginfo)`

The `protected` entry point is optional. This is invoked when another PD calls `microkit_ppcall` on a channel shared with the PD.

The `ch` argument identifies the channel on which the PP was invoked. Indirectly this identifies the PD performing the call. Channel identifiers are specified in the system configuration. **Note:** The channel argument is passed by the system and is unforgeable.

The `msginfo` argument is the argument passed to the PP and is provided by the calling PD. The contents of the message is up to a pre-arranged protocol between the PDs. The message contents are opaque to the system. Note: The message is *copied* from the caller.

The returned `microkit_msginfo` is the return value of the protected procedure. As with arguments, this is *copied* to the caller.

5.4 `seL4_Bool fault(microkit_child child, microkit_msginfo msginfo, microkit_msginfo *reply_msginfo)`

The `fault` entry point being invoked depends on whether the given PD has children. It is invoked when a child PD or VM causes a fault.

The `child` argument identifies the child that caused the fault.

The `msginfo` argument is given by the seL4 kernel when a fault occurs and contains information as to what fault occurred.

The `reply_msginfo` argument is given by libmicrokit and can be used to reply to the fault.

The returned `seL4_Bool` is whether or not to reply to the fault with the message `reply_msginfo`. Returning `seL4_True` will reply to the fault. Returning `seL4_False` will not reply to the fault.

You can use `microkit_msginfo_get_label` on `msginfo` to deduce what kind of fault happened (for example, whether it was a user exception or a virtual memory fault).

Whether or not you reply to the fault depends on the type of fault that has occurred and how you want to handle it.

To find the full list of possible faults that could occur and details regarding to replying to a particular kind of fault, please see the ‘Faults’ section of the [seL4 reference manual](#).

5.5 `microkit_msginfo microkit_ppcall(microkit_channel ch, microkit_msginfo msginfo)`

Performs a call to a protected procedure in a different PD. The `ch` argument identifies the protected procedure to be called. `msginfo` is passed as argument to the protected procedure. Channel identifiers are specified in the system configuration.

The protected procedure’s return data is returned in the `microkit_msginfo`.

5.6 `void microkit_notify(microkit_channel ch)`

Notify the channel `ch`. Channel identifiers are specified in the system configuration.

5.7 void microkit_irq_ack(microkit_channel ch)

Acknowledge the interrupt identified by the specified channel.

5.8 void microkit_deferred_notify(microkit_channel ch)

The same as `microkit_notify` but will instead not actually perform the notify until the entry point where `microkit_deferred_notify` was called returns.

It is important to note that only a single 'deferred' API call can be made within the same entry point.

The purpose of this API is for performance critical code as this API saves a kernel system call.

5.9 void microkit_deferred_irq_ack(microkit_channel ch)

The same as `microkit_irq_ack` but will instead not actually perform the IRQ acknowledge until the entry point where `microkit_deferred_irq_ack` was called returns.

It is important to note that only a single 'deferred' API call can be made within the same entry point.

The purpose of this API is for performance critical code as this API saves a kernel system call.

5.10 void microkit_pd_restart(microkit_child pd, uintptr_t entry_point)

Restart the execution of a child protection domain with ID `pd` at the given `entry_point`. This will set the program counter of the child protection domain to `entry_point`.

5.11 void microkit_pd_stop(microkit_child pd)

Stop the execution of the child protection domain with ID `pd`.

5.12 microkit_msginfo microkit_msginfo_new(uint64_t label, uint16_t count)

Creates a new message structure.

The message can be passed to `microkit_ppcall` or returned from `protected`.

5.13 uint64_t microkit_msginfo_get_label(microkit_msginfo msginfo)

Returns the label from a message.

5.14 uint64_t microkit_msginfo_get_count(microkit_msginfo msginfo)

Returns the count of message registers in the message.

5.15 uint64_t microkit_mr_get(uint8_t mr)

Get a message register.

5.16 void microkit_mr_set(uint8_t mr, uint64_t value)

Set a message register.

5.17 `void microkit_vcpu_restart(microkit_child vcpu, seL4_Word entry_point)`

Restart the execution of a VM's virtual CPU with ID `vcpu` at the given entry point. This will set the program counter of the vCPU to `entry_point`.

5.18 `void microkit_vcpu_stop(microkit_child vcpu)`

Stop the execution of the VM's virtual CPU with ID `vcpu`.

5.19 `void microkit_vcpu_arm_inject_irq(microkit_child vcpu, seL4_Uint16 irq, seL4_Uint8 priority, seL4_Uint8 group, seL4_Uint8 index)`

Inject a virtual ARM interrupt for a virtual CPU `vcpu` with IRQ number `irq`. The priority (0-31) determines what ARM GIC (generic interrupt controller) priority level the virtual IRQ will be injected as. The `group` determines whether the virtual IRQ will be injected into secure world (1) or non-secure world (0). The `index` is the index of the virtual GIC list register.

5.20 `void microkit_vcpu_arm_ack_vppei(microkit_child vcpu, seL4_Word irq)`

Acknowledge a ARM virtual Private Peripheral Interrupt (PPI) with IRQ number `irq` for a VM's vCPU with ID `vcpu`.

5.21 `seL4_Word microkit_vcpu_arm_read_reg(microkit_child vcpu, seL4_Word reg)`

Read a register for a given virtual CPU with ID `vcpu`. The `reg` argument is the index of the register that is read. The list of registers is defined by the enum `seL4_VCPUReg` in the seL4 source code.

5.22 `void microkit_vcpu_arm_write_reg(microkit_child vcpu, seL4_Word reg, seL4_Word value)`

Write to a register for a given virtual CPU with ID `vcpu`. The `reg` argument is the index of the register that is written to. The `value` argument is what the register will be set to. The list of registers is defined by the enum `seL4_VCPUReg` in the seL4 source code.

6 System Description File

This section describes the format of the System Description File (SDF).

The system description file is an XML file that is provided as input to the `microkit` tool.

The root element of the XML file is `system`.

Within the `system` root element the following child elements are supported:

- `protection_domain`
- `memory_region`
- `channel`

6.1 `protection_domain`

The `protection_domain` element describes a protection domain.

It supports the following attributes:

- `name`: A unique name for the protection domain
- `pp`: (optional) Indicates that the protection domain has a protected procedure; defaults to `false`.
- `priority`: The priority of the protection domain (integer 0 to 254).
- `budget`: (optional) The PD's budget in microseconds; defaults to 1,000.
- `period`: (optional) The PD's period in microseconds; must not be smaller than the budget; defaults to the budget.
- `passive`: (optional) Indicates that the protection domain will be passive and thus have its scheduling context removed after initialisation; defaults to `false`.
- `stack_size`: (optional) Number of bytes that will be used for the PD's stack. Must be between 4KiB and 16MiB and be 4K page-aligned. Defaults to 4KiB.

Additionally, it supports the following child elements:

- `program_image`: (exactly one) Describes the program image for the protection domain.
- `map`: (zero or more) Describes mapping of memory regions into the protection domain.
- `irq`: (zero or more) Describes hardware interrupt associations.
- `setvar`: (zero or more) Describes variable rewriting.
- `protection_domain`: (zero or more) Describes a child protection domain.
- `virtual_machine`: (zero or one) Describes a child virtual machine.

The `program_image` element has a single `path` attribute describing the path to an ELF file.

The `map` element has the following attributes:

- `mr`: Identifies the memory region to map.
- `vaddr`: Identifies the virtual address at which to map the memory region.
- `perms`: Identifies the permissions with which to map the memory region. Can be a combination of `r` (read), `w` (write), and `x` (eXecute), with the exception of a write-only mapping (just `w`).
- `cached`: (optional) Determines if mapped with caching enabled or disabled. Defaults to `true`.
- `setvar_vaddr`: (optional) Specifies a symbol in the program image. This symbol will be rewritten with the virtual address of the memory region.

The `irq` element has the following attributes:

- `irq`: The hardware interrupt number.

- **id:** The channel identifier. Must be at least 0 and less than 63.
- **trigger:** (optional) Whether the IRQ is edge triggered ("edge") or level triggered ("level"). Defaults to "level".

The `setvar` element has the following attributes:

- **symbol:** Name of a symbol in the ELF file.
- **region_paddr:** Name of an MR. The symbol's value shall be updated to this MR's physical address.

The `protection_domain` element has the same attributes as any other protection domain as well as:

- **id:** The ID of the child for the parent to refer to.

The `virtual_machine` element has the following attributes:

- **name:** A unique name for the virtual machine
- **priority:** The priority of the virtual machine (integer 0 to 254).
- **budget:** (optional) The VM's budget in microseconds; defaults to 1,000.
- **period:** (optional) The VM's period in microseconds; must not be smaller than the budget; defaults to the budget.

Additionally, it supports the following child elements:

- **vcpu:** (exactly one) Describes the virtual CPU that will be tied to the virtual machine. At the moment only one vCPU is supported and so only one of these elements can exist for a virtual machine.
- **map:** (zero or more) Describes mapping of memory regions into the virtual machine.

The `vcpu` element has a single `id` attribute defining the identifier used for the virtual machine's vCPU.

The `map` element has the same attributes as the protection domain with the exception of `setvar_vaddr`.

6.2 memory_region

The `memory_region` element describes a memory region.

It supports the following attributes:

- **name:** A unique name for the memory region
- **size:** Size of the memory region in bytes (must be a multiple of the page size)
- **page_size:** (optional) Size of the pages used in the memory region; must be a supported page size if provided. Defaults to the smallest page size for the target architecture.
- **phys_addr:** (optional) The physical address for the start of the memory region (must be a multiple of the page size).

The `memory_region` element does not support any child elements.

6.2.1 Page sizes by architecture

Below are the available page sizes for each architecture that Microkit supports.

6.2.1.1 AArch64

- 0x1000 (4KiB)
- 0x200000 (2MiB)

6.2.1.2 RISC-V 64-bit

- 0x1000 (4KiB)
- 0x200000 (2MiB)

6.3 channel

The `channel` element has exactly two `end` children elements for specifying the two PDs associated with the channel.

The `end` element has the following attributes:

- `pd`: Name of the protection domain for this end.
- `id`: Channel identifier in the context of the named protection domain. Must be at least 0 and less than 63.

The `id` is passed to the PD in the `notified` and `protected` entry points. The `id` should be passed to the `microkit_notify` and `microkit_ppcall` functions.

7 Board Support Packages

This chapter describes the board support packages that are available in the SDK.

7.1 i.MX8MM-EVK

Microkit produces a raw binary file, so when using U-Boot you must execute the image using:

```
=> go 0x41000000
```

7.2 i.MX8MQ-EVK

Microkit produces a raw binary file, so when using U-Boot you must execute the image using:

```
=> go 0x41000000
```

7.3 MaaXBoard

The MaaXBoard is a low-cost ARM SBC based on the NXP i.MX8MQ system-on-chip.

Microkit produces a raw binary file, so when using U-Boot you must execute the image using:

```
=> go 0x50000000
```

7.4 Odroid-C2

The HardKernel Odroid-C2 is an ARM SBC based on the Amlogic Meson S905 system-on-chip. It should be noted that the Odroid-C2 is no longer available for purchase but its successor, the Odroid-C4, is readily available at the time of writing.

Microkit produces a raw binary file, so when using U-Boot you must execute the image using:

```
=> go 0x20000000
```

7.5 Odroid-C4

The HardKernel Odroid-C4 is an ARM SBC based on the Amlogic Meson S905X3 system-on-chip.

Microkit produces a raw binary file, so when using U-Boot you must execute the image using:

```
=> go 0x20000000
```

7.6 TQMa8XQP 1GB

The TQMa8XQP is a system-on-module designed by TQ-Systems GmbH. The module incorporates an NXP i.MX8X Quad Plus system-on-chip and 1GiB ECC memory.

TQ-Systems provide the MBa8Xx carrier board for development purposes. The instructions provided assume the use of the MBa8Xx carrier board. If you are using a different carrier board please refer to the appropriate documentation.

Note: There are different configurations of the TQMa8Xx board which include different NXP SoCs and different memory configurations. Such modules are not supported.

The MBa8Xx provides access to the TQMa8XQP UART via UART-USB bridge. To access the UART connect a USB micro cable to port **X13**. The UART-USB bridge supports 4 individual UARTs; the UART is connected to the 2nd port.

By default the SoM will autoboot using U-Boot. Hit any key during the boot process to stop the autoboot.

A new board will autoboot to Linux. You will likely want to disable autoboot:

```
=> env set bootdelay -1
=> env save
```

The board can be reset by pressing switch **S4** (located next to the Ethernet port). Alternatively, you can use the `reset` command from the U-Boot prompt.

During development the most convenient way to boot a Microkit image is via network booting. U-Boot support booting via the *tftp* protocol. To support this you'll want to configure the network. U-Boot supports DHCP, however it is often more reliable to explicitly set an IP address. For example:

```
=> env set ipaddr 10.1.1.2
=> env set netmask 255.255.255.0
=> env set serverip 10.1.1.1
=> env save
```

To use *tftp* you also need to set the file to load and the memory address to load it to:

```
=> env set bootfile loader.img
=> env set loadaddr 0x80280000
=> env save
```

The system image generated by the Microkit tool is a raw binary file.

An example sequence of commands for booting is:

```
=> tftpboot
=> dcache flush
=> icache flush
=> go ${loadaddr}
```

Rather than typing these each time you can create a U-Boot script:

```
=> env set microkit 'tftpboot; dcache flush; icache flush; go ${loadaddr}'
=> env save
=> run microkit
```

When debugging is enabled the kernel will use the same UART as U-Boot.

7.7 QEMU virt (AArch64)

Support is available for the virtual AArch64 QEMU platform. This is a platform that is not based on any specific SoC or hardware platform and is intended for simulating systems for development or testing.

It should be noted that the platform support is configured with 2GB of main memory and a single Cortex-A53 CPU.

You can use the following command to simulate a Microkit system:

```
$ qemu-system-aarch64 \
  -machine virt,virtualization=on \
  -cpu cortex-a53 \
  -nographic \
  -serial mon:stdio \
  -device loader,file=[SYSTEM IMAGE],addr=0x70000000,cpu-num=0 \
  -m size=2G
```

You can find more about the QEMU virt platform in the [QEMU documentation](#).

7.8 QEMU virt (RISC-V 64-bit)

Support is available for the virtual RISC-V (64-bit) QEMU platform. This is a platform that is not based on any specific SoC or hardware platform and is intended for simulating systems for development or testing.

It should be noted that the platform support is configured with 2GB of main memory.

You can use the following command to simulate a Microkit system:

```
$ qemu-system-riscv64 \
  -machine virt \
  -nographic \
  -serial mon:stdio \
  -kernel [SYSTEM IMAGE] \
  -m size=2G
```

QEMU will start the system image using its packaged version of OpenSBI.

You can find more about the QEMU virt platform in the [QEMU documentation](#).

7.9 Pine64 Star64

Support is available for the Pine64 Star64 platform which is based on the StarFive JH7110 SoC.

The platform has a 4GB and 8GB model, we assume the 4GB model.

The default boot flow of the Star64 is: 1. OpenSBI 2. U-Boot 3. Operating System

This means that the system image that Microkit produces does not need to be explicitly packaged with an SBI implementation such as OpenSBI.

To execute the system image produced by Microkit, execute the following command in U-Boot:

```
=> go 0x60000000
```

7.10 ZCU102

Initial support is available for the Xilinx ZCU102.

FIXME: Additional documentation required here.

The ZCU102 can run on a physical board or on an appropriate QEMU based emulator.

Microkit produces a raw binary file, so when using U-Boot you must execute the image using:

```
=> go 0x40000000
```

For simulating the ZCU102 using QEMU, use the following command:

```
$ qemu-system-aarch64 \
  -m size=4G \
  -machine xlnx-zcu102,virtualization=on \
  -nographic \
  -device loader,file=[SYSTEM IMAGE],addr=0x40000000,cpu-num=0 \
  -serial mon:stdio
```


It should be noted that when using U-Boot to load and run a Microkit system image, that there may be additional setup needed.

For the ZynqMP class of platforms, which the ZCU102 is apart of, U-Boot does not start the Microkit system Exception Level 2 (EL2) which is necessary for Microkit to start (this is because seL4 is configured as a hypervisor).

You can see that when using the `go` command, U-Boot is [unconditionally always dropping down to EL1](#).

To avoid this behaviour, the call to `armv8_switch_to_el1` should be replaced with `armv8_switch_to_el2` in this `do_go_exec` function.

7.11 Adding Platform Support

The following section is a guide for adding support for a new platform to Microkit.

7.11.1 Prerequisites

Before you can start with adding platform support to Microkit, the platform must be supported by the seL4 kernel. You can find information on how to do so [here](#).

7.11.2 Getting Microkit Components Working

The first step to adding Microkit support is to modify the `build_sdk.py` script in order to build the required artefacts for the new platform. This involves adding to the `SUPPORTED_BOARDS` list with the `BoardInfo` options containing the platform specific attributes. This should be fairly self-explanatory by looking at the existing entries with the exception of the `loader_link_address`.

The `loader_link_address` parameter specifies the physical address of where the bootloader for Microkit (which is responsible for setting up the system before seL4 starts) is going to be loaded. This address needs to match where in main memory the final system image is actually loaded (e.g where a previous bootloader such as U-Boot loads the image to). This means that the address is restricted to the platform's main memory region.

The other component of Microkit that is platform dependent is the loader itself. The loader will attempt to access the UART for debug output which requires a basic `putc` implementation. The UART device used in the loader should be the same as what is used for the seL4 kernel debug output.

It should be noted that on RISC-V platforms, the SBI will be used for `putc` so no porting is necessary.

Once you have patched the loader and the SDK build script, there should be no other changes required to have a working platform port. It is a good idea at this point to boot a hello world system to confirm the port is working.

If there are issues with porting the platform, please [open an issue on GitHub](#).

7.11.3 Contributing Platform Support

Once you believe that the port works, you can [open a pull request](#) with required changes as well as documentation in the manual about the platform and how to run Microkit images on it.

8 Rationale

This section describes the rationales driving the Microkit design choices.

8.1 Overview

The seL4 microkernel provides a set of powerful and flexible mechanisms that can be used for building almost arbitrary systems. While minimising constraints on the nature of system designs and scope of deployments, this flexibility makes it challenging to design the best system for a particular use case, requiring extensive seL4 experience from developers.

The Microkit addresses this challenge by constraining the system architecture to one that provides enough features and power for its target usage class (IoT, cyberphysical and other embedded systems with a static architecture), enabling a much simpler set of developer-visible abstractions.

8.2 Protection Domains

PDs are single-threaded to keep the programming model and implementations simple, and because this serves the needs of most present use cases in the target domains. Extending the model to multithreaded applications (clients) is straightforward and can be done if needed. Extending to multithreaded services is possible but requires additional infrastructure for which we see no need in the near future.

8.3 Protected Procedure Priorities

The restriction of only calling to higher priority prevents deadlocks and reflects the notion that the callee operates on behalf of the caller, and it should not be possible to preempt execution of the callee unless the caller could be preempted as well.

This greatly simplifies reasoning about real-time properties in the system; in particular, it means that PPs can be used to implement *resource servers*, where shared resources are encapsulated in a component that ensures mutual exclusion, while avoiding unbounded priority inversions through the *immediate priority ceiling protocol*.

While it would be possible to achieve the same by allowing PPs between PDs of the same priority, this would be much harder to statically analyse for loop-freedom (and thus deadlock-freedom). The drawback is that we waste a part of the priority space where a logical entity is split into multiple PDs, eg to separate out a particularly critical component to formally verify it, when the complete entity would be too complex for formal verification. For the kinds of systems targeted by the Microkit, this reduction of the usable priority space is unlikely to cause problems.

8.4 Protected Procedure Argument Size

The limitation on the size of by-value arguments is forced by the (architecture-dependent) limits on the payload size of the underlying seL4 operations, as well as by efficiency considerations. The protected procedure payload should be considered as analogous to function arguments in the C language; similar limitations exist in the C ABIs (Application Binary Interfaces) of various platforms.

8.5 Limits

The limitation on the number of protection domains in the system is relatively arbitrary. Based on experience with the system and the types of systems being built it is possible for this to be increased in the future.

The limitation on the number of channels for a protection domain is based on the size of the notification word in seL4. Changing this to be larger than 64 would most likely require changes to seL4. The reason for why the limit is not a power of two is due to part of the notification word being for internal libmicrokit use.